

# React Native Core Components Masterclass

Detailed Technical Guide, Native Bridge Mechanics & Comprehensive Code Examples

## □ Executive Summary & Architecture Overview

In React Native, UI components are not HTML elements rendered inside a webview. Instead, every React Native core component acts as a lightweight JavaScript wrapper around a real, native platform view (iOS UIKit or Android View/ViewGroup). React's reconciliation algorithm passes layout calculations (computed via Facebook's C-based 'Yoga' Flexbox engine) over the bridge/JSI to assemble a native view tree.

## Core Components Architectural Mapping

| React Native Component   | iOS Native Counterpart          | Android Native Counterpart                |
|--------------------------|---------------------------------|-------------------------------------------|
| View                     | UIView                          | android.view.ViewGroup                    |
| Text                     | UILabel                         | android.widget.TextView                   |
| Image                    | UIImageView                     | android.widget.ImageView                  |
| Button                   | UIButton                        | android.widget.Button                     |
| TextInput                | UITextField / UITextView        | android.widget.EditText                   |
| ScrollView               | UIScrollView                    | android.widget.ScrollView                 |
| FlatList                 | RCTCollectionView / UITableView | androidx.recyclerview.widget.RecyclerView |
| SafeAreaView / StatusBar | UIWindow / SafeAreaInsets       | android.view.Window / SystemUI            |

## 1. View Component

The View component is the fundamental building block of any React Native user interface. It maps directly to a native container (UIView on iOS and android.view.ViewGroup on Android). A View is designed to hold, organize, and style child elements using Flexbox layout.

### Technical Concepts & Architecture

- **Yoga Flexbox Layout:** Every View is calculated by the Yoga layout engine using C++ before being serialized to native view nodes.
- **Default Flex Direction:** Unlike Web Flexbox where `flexDirection` defaults to `row`, React Native defaults `flexDirection` to `column`. All layout dimensions are unitless (representing density-independent pixels / dip).

- **No Raw Text Rule:** A View cannot directly contain raw string nodes. Writing `<View>Hello</View>` will immediately throw a render error (``Text strings must be rendered within a <Text> component``).
- **Touch & Pointer System:** Supports touch event handling (`onStartShouldSetResponder`), slop regions (`hitSlop`), pointer pass-through (`pointerEvents`), and accessibility attributes.

## Key Props

- **style:** Accepts style objects or arrays of style objects (e.g., `flex`, `backgroundColor`, `margin`, `padding`, `borderRadius`, `shadowColor`).
- **onLayout:** Fires when the native view layout is calculated or changes, providing `{ x, y, width, height }` frame dimensions.
- **pointerEvents:** Controls touch handling. Values: `auto`, `none`, `box-none`, `box-only`. Useful for overlays.
- **hitSlop:** Extends the touchable area beyond the view's physical bounds without changing visual layout (e.g., `{ top: 10, bottom: 10, left: 10, right: 10 }`).

### □ View Example 1: Card Container with Native Elevation & Shadows

```
import React from 'react';
import { View, Text, StyleSheet } from 'react-native';

export default function BasicViewCard() {
  return (
    <View style={styles.cardContainer}>
      <View style={styles.cardHeader}>
        <Text style={styles.headerTitle}>User Card</Text>
      </View>
      <View style={styles.cardBody}>
        <Text style={styles.bodyText}>Name: Alex Johnson</Text>
        <Text style={styles.bodyText}>Role: Mobile Developer</Text>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  cardContainer: {
    backgroundColor: 'ffffff',
    borderRadius: 12,
    padding: 16,
    margin: 16,
    // Cross-Platform Shadows
    shadowColor: 'black',
    shadowOffset: { width: 0, height: 4 },
    shadowOpacity: 0.1,
    shadowRadius: 6,
    elevation: 5, // Android shadow elevation
  },
  cardHeader: {
    borderBottomWidth: 1,
    borderBottomColor: 'eeeeee',
    paddingBottom: 8,
    marginBottom: 12,
  },
  headerTitle: {
    fontSize: 18,
    fontWeight: 'bold',
    color: '#1e3a8a',
  },
  bodyText: {
    color: '#1e3a8a',
  },
});
```

```

    cardBody: {
      gap: 6,
    },
    bodyText: {
      fontSize: 14,
      color: '#333333',
    },
  },
});

```

This example demonstrates how View components are nested to build structured cards. Note how `shadowColor`, `shadowOffset`, `shadowOpacity`, and `shadowRadius` apply iOS native layer shadows, while `elevation` applies Android elevation shading.

#### □ View Example 2: Screen Sectioning using Flexbox Layout

```

import React from 'react';
import { View, Text, StyleSheet } from 'react-native';

export default function FlexLayoutExample() {
  return (
    <View style={styles.screenContainer}>
      /* Top Navigation Header */
      <View style={styles.topBar}>
        <Text style={styles.topBarText}>Left Action</Text>
        <Text style={styles.topBarTitle}>Dashboard</Text>
        <Text style={styles.topBarText}>Right Action</Text>
      </View>

      /* Central Content Area taking up remaining space */
      <View style={styles.contentBody}>
        <View style={styles.gridBox}><Text>Box 1</Text></View>
        <View style={styles.gridBox}><Text>Box 2</Text></View>
      </View>

      /* Bottom Sticky Footer */
      <View style={styles.footerBar}>
        <Text style={styles.footerText}>Sticky Screen Footer</Text>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  screenContainer: {
    flex: 1, // Fills full available screen height
    backgroundColor: '#f8fafc',
  },
  topBar: {
    height: 60,
    backgroundColor: '#1e3a8a',
    flexDirection: 'row', // Horizontal axis
    justifyContent: 'space-between',
    alignItems: 'center',
    paddingHorizontal: 16,
  },
  topBarTitle: {
    color: 'ffffff',
    fontSize: 18,
    fontWeight: 'bold',
  },
  topBarText: {

```

```

    color: '#93c5fd',
    fontSize: 14,
  },
  contentBody: {
    flex: 1, // Expands to fill middle space
    flexDirection: 'row',
    justifyContent: 'space-around',
    alignItems: 'center',
    padding: 20,
  },
  gridBox: {
    width: 120,
    height: 120,
    backgroundColor: '#e2e8f0',
    borderRadius: 8,
    justifyContent: 'center',
    alignItems: 'center',
  },
  footerBar: {
    height: 50,
    backgroundColor: '#0f172a',
    justifyContent: 'center',
    alignItems: 'center',
  },
  footerText: {
    color: 'ffffff',
    fontSize: 13,
  },
});

```

The `flex: 1` on `screenContainer` occupies the entire viewport height. Setting `flexDirection: 'row'` on `topBar` aligns the three text children horizontally across the main axis, while `justifyContent: 'space-between'` distributes them across the edges.

#### □ View Example 3: Absolute Positioning & Stacking Overlays

```

import React from 'react';
import { View, Text, StyleSheet } from 'react-native';

export default function AbsoluteOverlayView() {
  return (
    <View style={styles.container}>
      <View style={styles.bannerContainer}>
        <Text style={styles.bannerText}>Product Display Card</Text>

        {/* Absolute Positioned Badge */}
        <View style={styles.badgeOverlay}>
          <Text style={styles.badgeText}>SALE -20%</Text>
        </View>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
    justifyContent: 'center',
  },
  bannerContainer: {

```

```

    height: 180,
    backgroundColor: '#3b82f6',
    borderRadius: 12,
    justifyContent: 'center',
    alignItems: 'center',
    position: 'relative', // Context for absolute children
  },
  bannerText: {
    color: 'ffffff',
    fontSize: 20,
    fontWeight: 'bold',
  },
  badgeOverlay: {
    position: 'absolute',
    top: 12,
    right: 12,
    backgroundColor: '#ef4444',
    paddingHorizontal: 10,
    paddingVertical: 4,
    borderRadius: 20,
    zIndex: 10, // Higher stacking index
  },
  badgeText: {
    color: 'ffffff',
    fontSize: 12,
    fontWeight: 'bold',
  },
});

```

In React Native, setting `position: 'absolute'` removes the View from the normal Yoga layout flow. Properties like `top`, `right`, `bottom`, `left`, and `zIndex` position the badge relative to its nearest relative parent.

## 2. Text Component

The Text component is the only built-in React Native element dedicated to displaying written characters. Unlike the web, raw text cannot exist as a direct child of a View container; it must always be wrapped in a `<Text>` component.

### Technical Concepts & Architecture

- **Native Rendering:** Maps directly to native text view engines (`UILabel` / `NSTextStorage` on iOS and `android.widget.TextView` on Android).
- **Inline Nesting & Inheritance:** When a `<Text>` component is nested inside another `<Text>` component, React Native treats it as an inline formatting span (similar to `<span>` inside `<p>`). Inner text components inherit font family, color, and size from their parent text block.
- **Native Truncation:** Text handles truncation natively. The `numberOfLines` prop limits rendering lines, and `ellipsizeMode` controls where truncation dots (...) appear.
- **Dynamic Type Accessibility:** Respects user system accessibility settings for font scaling unless `allowFontScaling={false}` is explicitly declared.

### Key Props

- **numberOfLines:** Limits max rendering lines before clipping text.
- **ellipsizeMode:** Specifies truncation point (`'tail'`, `'head'`, `'middle'`, `'clip'`). Default is `'tail'`.

- **onPress:** Makes the text block interactive without needing a View wrapper.
- **selectable:** Allows mobile users to highlight and copy text via long-press (defaults to `false`).
- **allowFontScaling:** Prevents or enables automatic text resizing when users change OS font size accessibility settings.

#### □ Text Example 1: Typography Hierarchy & Nested Text Formatting

```
import React from 'react';
import { View, Text, StyleSheet } from 'react-native';

export default function TypographyHierarchy() {
  return (
    <View style={styles.container}>
      <Text style={styles.heading}>Main Section Heading</Text>
      <Text style={styles.subheading}>Subheading with secondary styling</Text>

      { /* Nested Text Inheritance Example */ }
      <Text style={styles.paragraph}>
        This is a paragraph where <Text style={styles.boldSpan}>important words</Text> are
        highlighted inline, and <Text style={styles.italicSpan}>italicized notes</Text> follow
        seamlessly.
      </Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { padding: 20, backgroundColor: 'ffffff' },
  heading: { fontSize: 24, fontWeight: 'bold', color: '#0f172a', marginBottom: 6 },
  subheading: { fontSize: 16, color: '#64748b', marginBottom: 16 },
  paragraph: { fontSize: 14, color: '#334155', lineHeight: 22 },
  boldSpan: { fontWeight: 'bold', color: '#1e3a8a' },
  italicSpan: { fontStyle: 'italic', color: '#0284c7' },
});
```

The paragraph demonstrates inline nesting. Inside `

#### □ Text Example 2: Dynamic Text Truncation with Read-More Toggle

```
import React, { useState } from 'react';
import { View, Text, StyleSheet, TouchableOpacity } from 'react-native';

export default function ExpandableText() {
  const [expanded, setExpanded] = useState(false);

  const fullBio = "React Native combines the best parts of native development with React, a best-in-class JavaScript library for building user interfaces. Use a little or a lot. You can use React Native today in your existing Android and iOS projects or you can create a whole new app from scratch.";

  return (
    <View style={styles.container}>
      <Text
        style={styles.bioText}
        numberOfLines={expanded ? undefined : 2}
        ellipsizeMode="tail"
      >
        {fullBio}
      </Text>
    </View>
  );
}
```

```

    </Text>

    <TouchableOpacity onPress={() => setExpanded(!expanded)}>
      <Text style={styles.toggleButton}>
        {expanded ? 'Read Less ▲' : 'Read More ▼'}
      </Text>
    </TouchableOpacity>
  </View>
);
}

const styles = StyleSheet.create({
  container: { padding: 16, backgroundColor: '#f8fafc', borderRadius: 8 },
  bioText: { fontSize: 14, lineHeight: 20, color: '#1e293b' },
  toggleButton: { marginTop: 8, color: '#2563eb', fontWeight: 'bold', fontSize: 13 },
});

```

Setting `numberOfLines={expanded ? undefined : 2}` dynamically toggles native truncation. When `expanded` is false, the native UILabel/TextView clips text after line 2 and inserts `...` at the end (`ellipsizeMode='tail').

### □ Text Example 3: Interactive Inline Text Links with Linking API

```

import React from 'react';
import { View, Text, StyleSheet, Linking, Alert } from 'react-native';

export default function InteractiveTextLinks() {
  const handleOpenLink = async (url) => {
    const supported = await Linking.canOpenURL(url);
    if (supported) {
      await Linking.openURL(url);
    } else {
      Alert.alert('Error', `Cannot open URL: ${url}`);
    }
  };

  return (
    <View style={styles.container}>
      <Text style={styles.body}>
        By clicking Register, you agree to our{' '}
        <Text
          style={styles.linkText}
          onPress={() => handleOpenLink('https://reactnative.dev/terms')}
        >
          Terms of Service
        </Text>{' '}
        and{' '}
        <Text
          style={styles.linkText}
          onPress={() => handleOpenLink('https://reactnative.dev/privacy')}
        >
          Privacy Policy
        </Text>.
      </Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { padding: 20 },
  body: { fontSize: 14, color: '#475569', lineHeight: 22 },
});

```

```
linkText: { color: '#2563eb', textDecorationLine: 'underline', fontWeight: '600' },
});
```

The `Text` component supports an `onPress` prop directly. This allows embedding clickable inline text links directly within continuous body text without breaking paragraph flow.

## 3. Image Component

The Image component displays local static graphics or remote network images. Under the hood, iOS renders `UIImageView` and Android renders `android.widget.ImageView` (or uses native image loaders like Glide/Fresco).

### Technical Concepts & Architecture

- **Local vs Remote Image Bundling:** Local images imported via `require('./logo.png')` are bundled at build time. Dimensions are known automatically. Remote network images (`{ uri: 'https://...' }`) MUST have explicit CSS `width` and `height` declared, or they render with 0x0 size.
- **Resize Modes:** Dictates how images scale into container frames (`'cover'`, `'contain'`, `'stretch'`, `'repeat'`, `'center'`).
- **Caching Mechanism:** Remote images use disk and memory caching automatically on native platforms.
- **Network Lifecycle Events:** The Image component exposes event hooks (`onLoadStart`, `onLoadEnd`, `onError`) to manage fallback UI when images fail to download.

### Key Props

- **source:** The asset reference. Either `require('path')` or `{ uri: 'https://...' }` object.
- **resizeMode:** Determines image layout fit (`'cover'`, `'contain'`, `'stretch'`, `'repeat'`, `'center'`).
- **onLoadStart:** Fires as soon as network downloading begins.
- **onLoad:** Fires when image loading completes successfully.
- **onError:** Fires when network fails or URL returns a 404/500 code.

#### Image Example 1: Local vs Remote Images & ResizeMode Fits

```
import React from 'react';
import { View, Text, Image, StyleSheet, ScrollView } from 'react-native';

export default function ResizeModeComparison() {
  const remoteUri = 'https://picsum.photos/300/200';

  return (
    <ScrollView style={styles.container}>
      <Text style={styles.title}>ResizeMode Comparison</Text>

      <Text style={styles.label}>1. cover (Fills frame, crops overflow)</Text>
      <Image source={{ uri: remoteUri }} style={styles.imgFrame} resizeMode="cover" />

      <Text style={styles.label}>2. contain (Scales uniformly, no cropping)</Text>
      <Image source={{ uri: remoteUri }} style={styles.imgFrame} resizeMode="contain" />

      <Text style={styles.label}>3. stretch (Distorts aspect ratio to fit exact frame)</Text>
      <Image source={{ uri: remoteUri }} style={styles.imgFrame} resizeMode="stretch" />
    </ScrollView>
  );
}
```



```

    );
  }

  const styles = StyleSheet.create({
    container: { padding: 16, backgroundColor: 'ffffff' },
    title: { fontSize: 20, fontWeight: 'bold', marginBottom: 16, color: '#1e3a8a' },
    label: { fontSize: 13, fontWeight: '600', color: '#475569', marginTop: 12, marginBottom: 4 },
    imgFrame: { width: '100%', height: 140, backgroundColor: '#e2e8f0', borderRadius: 8 },
  });

```

Notice how `resizeMode='cover'` scales the image uniformly to cover the entire container frame (cropping excess image parts), whereas `contain` scales the image to fit entirely inside the frame without cropping.

#### □ Image Example 2: Circular Avatar Profile with Border & Online Badge

```

import React from 'react';
import { View, Image, Text, StyleSheet } from 'react-native';

export default function CircularAvatarProfile() {
  return (
    <View style={styles.profileCard}>
      <View style={styles.avatarWrapper}>
        <Image
          source={{ uri: 'https://i.pravatar.cc/150?img=12' }}
          style={styles.avatarImage}
        />
        <View style={styles.onlineBadge} />
      </View>
      <Text style={styles.userName}>Sarah Jenkins</Text>
      <Text style={styles.userHandle}>@sarah_dev</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  profileCard: { alignItems: 'center', padding: 20, backgroundColor: '#f1f5f9', borderRadius: 12 },
  avatarWrapper: { position: 'relative', marginBottom: 12 },
  avatarImage: {
    width: 90,
    height: 90,
    borderRadius: 45, // Exact half of width/height creates a circle
    borderWidth: 3,
    borderColor: '#2563eb',
  },
  onlineBadge: {
    width: 20,
    height: 20,
    borderRadius: 10,
    backgroundColor: '#22c55e',
    position: 'absolute',
    bottom: 2,
    right: 2,
    borderWidth: 2,
    borderColor: 'ffffff',
  },
  userName: { fontSize: 18, fontWeight: 'bold', color: '#0f172a' },
  userHandle: { fontSize: 14, color: '#64748b' },
});

```

To make an Image perfectly circular in React Native, set `borderRadius` to exactly half of the `width` and `height` dimensions (`width: 90`, `height: 90`, `borderRadius: 45`).

### ❑ Image Example 3: Network Image with Loading Spinner & Error Fallback

```
import React, { useState } from 'react';
import { View, Image, ActivityIndicator, Text, StyleSheet } from 'react-native';

export default function NetworkImageWithFallback() {
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(false);

  return (
    <View style={styles.container}>
      <View style={styles.imageContainer}>
        <Image
          source={{ uri: error ? 'https://via.placeholder.com/200x150?text=Fallback' :
'https://picsum.photos/400/300' }}
          style={styles.image}
          onLoadStart={() => setLoading(true)}
          onLoadEnd={() => setLoading(false)}
          onError={() => {
            setError(true);
            setLoading(false);
          }}
        />

        {loading && (
          <View style={styles.spinnerOverlay}>
            <ActivityIndicator size="large" color="#2563eb" />
          </View>
        )}
      </View>
      {error && <Text style={styles.errorText}>Network image failed. Showing fallback.</Text>}
    </View>
  );
}

const styles = StyleSheet.create({
  container: { alignItems: 'center', padding: 20 },
  imageContainer: { width: 220, height: 160, borderRadius: 8, overflow: 'hidden', position:
'relative' },
  image: { width: '100%', height: '100%' },
  spinnerOverlay: {
    ...StyleSheet.absoluteFillObject, // Fills frame top:0, left:0, right:0, bottom:0
    backgroundColor: 'rgba(241, 245, 249, 0.8)',
    justifyContent: 'center',
    alignItems: 'center',
  },
  errorText: { color: '#ef4444', fontSize: 12, marginTop: 8 },
});
```

By capturing `onLoadStart`, `onLoadEnd`, and `onError`, this component renders an overlay `` while the image is downloading and switches to a fallback URL if network fetching fails.

## 4. Button Component

The Button component is a simple, minimalist native button element (`UIButton` on iOS and `android.widget.Button` on Android). While fast to implement for prototypes, real-world apps usually replace `` with custom `TouchableOpacity` or `Pressable` components due to severe styling constraints.

### Technical Concepts & Architecture

- **Styling Limitations:** The built-in Button component DOES NOT support style objects (`style={...}` prop is ignored). You cannot change font size, line height, padding, border radius, or shadows directly on a Button.
- **Platform Differences:** Setting `color='#2563eb'` changes the button background on Android, but changes text color on iOS.
- **Disabled State:** Disables user interaction and grays out the component natively.

### Key Props

- **title:** The plain text displayed inside the button (Required).
- **onPress:** Callback triggered when user taps the button.
- **color:** Platform-specific accent color (Text color on iOS; background color on Android).
- **disabled:** Boolean preventing tap events when true.

#### □ Button Example 1: Standard Native Buttons & Colors

```
import React from 'react';
import { View, Text, Button, Alert, StyleSheet } from 'react-native';

export default function BasicButtonExamples() {
  return (
    <View style={styles.container}>
      <Text style={styles.sectionTitle}>Standard Native Buttons</Text>

      <View style={styles.buttonWrapper}>
        <Button title="Default Button" onPress={() => Alert.alert('Pressed!')} />
      </View>

      <View style={styles.buttonWrapper}>
        <Button title="Colored Accent Button" color="#2563eb" onPress={() =>
Alert.alert('Colored!')} />
      </View>

      <View style={styles.buttonWrapper}>
        <Button title="Disabled Button" disabled={true} onPress={() => {}} />
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { padding: 20 },
  sectionTitle: { fontSize: 18, fontWeight: 'bold', marginBottom: 16 },
  buttonWrapper: { marginBottom: 12 },
});
```

Notice how native buttons render differently depending on OS. On iOS, buttons appear as clean flat text links, while on Android they render as material elevated buttons.

#### □ Button Example 2: Interactive Counter State Management

```
import React, { useState } from 'react';
import { View, Text, Button, StyleSheet } from 'react-native';

export default function InteractiveButtonCounter() {
  const [count, setCount] = useState(0);

  return (
    <View style={styles.card}>
      <Text style={styles.counterTitle}>Tap Counter: {count}</Text>

      <View style={styles.buttonRow}>
        <Button title="Increment (+1)" color="#16a34a" onPress={() => setCount(count + 1)} />
        <Button title="Reset" color="#dc2626" onPress={() => setCount(0)} />
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  card: { padding: 20, backgroundColor: '#f8fafc', borderRadius: 8, alignItems: 'center' },
  counterTitle: { fontSize: 20, fontWeight: 'bold', marginBottom: 16, color: '#0f172a' },
  buttonRow: { flexDirection: 'row', gap: 12 },
});
```

Connecting state updater functions (`setCount`) directly to the `onPress` prop causes React Native to re-render the screen UI instantly as state variables update.

#### □ Button Example 3: Custom Styled Button Alternative with TouchableOpacity

```
import React from 'react';
import { View, Text, TouchableOpacity, StyleSheet, Alert } from 'react-native';

export default function CustomStyledButtonAlternative() {
  return (
    <View style={styles.container}>
      <TouchableOpacity
        style={styles.customBtn}
        activeOpacity={0.8} // Dims button to 80% opacity on press
        onPress={() => Alert.alert('Custom Button Tapped!')}
      >
        <Text style={styles.customBtnText}>Custom Designed Button ★</Text>
      </TouchableOpacity>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { padding: 20 },
  customBtn: {
    backgroundColor: '#2563eb',
    paddingVertical: 14,
    paddingHorizontal: 24,
    borderRadius: 10,
    alignItems: 'center',
    shadowColor: '#2563eb',
    shadowOffset: { width: 0, height: 4 },
  },
  customBtnText: { color: 'white' },
});
```

```

    shadowOpacity: 0.3,
    shadowRadius: 8,
    elevation: 4,
  },
  customBtnText: { color: '#ffffff', fontSize: 16, fontWeight: 'bold' },
});

```

When production apps require custom fonts, rounded borders, icons, or shadows, developers replace ``<Button>`` with ``<TouchableOpacity>`` or ``<Pressable>``, which accept full CSS style objects.

## 5. TextInput Component

The `TextInput` component is the core element for taking keyboard input from users. It maps directly to `UITextField` / `UITextView` on iOS and `android.widget.EditText` on Android.

### Technical Concepts & Architecture

- **Controlled Components:** `TextInput` follows a two-way controlled component model (`value` bound to state, `onChangeText` updating state).
- **Native Keyboards:** Selecting `keyboardType` (`'email-address'`, `'numeric'`, `'phone-pad'`) brings up specialized native virtual keypads on mobile screens.
- **Security:** Hides characters for passwords (`secureTextEntry={true}`).
- **Keyboard Avoidance:** When focused, the soft keyboard opens and covers screen real estate. Use `KeyboardAvoidingView` to prevent keyboard from obscuring inputs.

### Key Props

- **value:** The current input text string stored in React state.
- **onChangeText:** Callback fired every time a key is pressed, returning the fresh string.
- **placeholder:** Placeholder text shown when input is empty.
- **secureTextEntry:** Masks text input with dots for passwords.
- **keyboardType:** Configures soft keyboard type (`'default'`, `'email-address'`, `'numeric'`, `'phone-pad'`).
- **multiline:** Enables multiline text entry.

#### □ TextInput Example 1: Basic Controlled Input with Live Feedback

```

import React, { useState } from 'react';
import { View, Text, TextInput, StyleSheet } from 'react-native';

export default function ControlledInputExample() {
  const [username, setUsername] = useState('');

  return (
    <View style={styles.container}>
      <Text style={styles.label}>Enter Username:</Text>
      <TextInput
        style={styles.inputField}
        placeholder="e.g. alex_dev"
        value={username}
        onChangeText={setUsername}
        autoCapitalize="none"
        autoCorrect={false}
      />
    </View>
  );
}

```

```

        <Text style={styles.previewText}>Live State Output: {username}</Text>
      </View>
    );
  }

const styles = StyleSheet.create({
  container: { padding: 20 },
  label: { fontSize: 14, fontWeight: '600', marginBottom: 6, color: '#3334155' },
  inputField: {
    height: 48,
    borderWidth: 1,
    borderColor: '#cbd5e1',
    borderRadius: 8,
    paddingHorizontal: 12,
    fontSize: 16,
    backgroundColor: '#ffffff',
  },
  previewText: { marginTop: 8, fontSize: 13, color: '#64748b', fontStyle: 'italic' },
});

```

This example illustrates two-way state binding. As the user types on the physical/virtual keyboard, `onChangeText={setUsername}` updates state, which immediately updates the `value={username}` prop.

#### □ TextInput Example 2: Form with Email Keyboard & Password Toggle

```

import React, { useState } from 'react';
import { View, Text, TextInput, TouchableOpacity, StyleSheet, Alert } from 'react-native';

export default function FormWithPasswordToggle() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [hidePassword, setHidePassword] = useState(true);

  const handleLogin = () => {
    if (!email.includes('@')) {
      Alert.alert('Validation Error', 'Please enter a valid email address.');
```

```

      return;
    }
    if (password.length < 6) {
      Alert.alert('Validation Error', 'Password must be at least 6 characters.');
```

```

      return;
    }
    Alert.alert('Success', 'Form submitted successfully!');
  };

  return (
    <View style={styles.formCard}>
      <Text style={styles.fieldLabel}>Email Address</Text>
      <TextInput
        style={styles.input}
        placeholder="user@example.com"
        keyboardType="email-address"
        autoCapitalize="none"
        value={email}
        onChangeText={setEmail}
      />

      <Text style={styles.fieldLabel}>Password</Text>
      <View style={styles.passwordRow}>
        <TextInput
          style={styles.passwordInput}

```

```

        placeholder="Min 6 characters"
        secureTextEntry={hidePassword}
        value={password}
        onChangeText={setPassword}
      />
      <TouchableOpacity onPress={() => setHidePassword(!hidePassword)}>
        <Text style={styles.toggleText}>{hidePassword ? 'Show' : 'Hide'}</Text>
      </TouchableOpacity>
    </View>

    <TouchableOpacity style={styles.submitBtn} onPress={handleLogin}>
      <Text style={styles.submitBtnText}>Sign In</Text>
    </TouchableOpacity>
  </View>
);
}

const styles = StyleSheet.create({
  formCard: { padding: 20, backgroundColor: '#ffffff', borderRadius: 12, borderWidth: 1,
borderColor: '#e2e8f0' },
  fieldLabel: { fontSize: 13, fontWeight: '600', color: '#475569', marginBottom: 6, marginTop:
12 },
  input: { height: 46, borderWidth: 1, borderColor: '#cbd5e1', borderRadius: 8,
paddingHorizontal: 12 },
  passwordRow: { flexDirection: 'row', alignItems: 'center', borderWidth: 1, borderColor:
'#cbd5e1', borderRadius: 8, paddingRight: 12 },
  passwordInput: { flex: 1, height: 46, paddingHorizontal: 12 },
  toggleText: { color: '#2563eb', fontWeight: 'bold', fontSize: 13 },
  submitBtn: { backgroundColor: '#1e3a8a', height: 48, borderRadius: 8, justifyContent:
'center', alignItems: 'center', marginTop: 20 },
  submitBtnText: { color: '#ffffff', fontWeight: 'bold', fontSize: 15 },
});

```

This form showcases keyboard type customization (`keyboardType='email-address'`) and dynamic password masking toggle using `secureTextEntry={hidePassword}`.

### □ TextInput Example 3: Auto-Formatting Phone Input & Multiline Area

```

import React, { useState } from 'react';
import { View, Text, TextInput, StyleSheet } from 'react-native';

export default function FormattingAndMultilineInput() {
  const [phone, setPhone] = useState('');
  const [bio, setBio] = useState('');

  const handlePhoneChange = (rawText) => {
    const cleaned = rawText.replace(/\D/g, ''); // Strip non-numeric
    let formatted = cleaned;
    if (cleaned.length > 6) {
      formatted = `(${cleaned.slice(0,3)}) ${cleaned.slice(3,6)}-${cleaned.slice(6,10)}`;
    } else if (cleaned.length > 3) {
      formatted = `(${cleaned.slice(0,3)}) ${cleaned.slice(3)}`;
    }
    setPhone(formatted);
  };

  return (
    <View style={styles.container}>
      <Text style={styles.label}>Phone Number (Auto-Formatted):</Text>
      <TextInput
        style={styles.input}

```

```

placeholder="(123) 456-7890"
keyboardType="phone-pad"
value={phone}
onChangeText={handlePhoneChange}
maxLength={14}
/>

<Text style={styles.label}>Bio / Feedback (Multiline):</Text>
<TextInput
  style={[styles.input, styles.multiline]}
  placeholder="Tell us about yourself..."
  multiline={true}
  numberOfLines={4}
  textAlignVertical="top" // Ensures text starts top-left on Android
  value={bio}
  onChangeText={setBio}
/>
</View>
);
}

const styles = StyleSheet.create({
  container: { padding: 20 },
  label: { fontSize: 14, fontWeight: '600', color: '#334155', marginTop: 12, marginBottom: 6 },
  input: { height: 46, borderWidth: 1, borderColor: '#cbd5e1', borderRadius: 8,
paddingHorizontal: 12 },
  multiline: { height: 100, paddingTop: 10 },
});

```

Notice `textAlignVertical='top'` on the multiline input. On Android, multiline text inputs default to vertical centering unless `textAlignVertical='top'` is specified.

## 6. ScrollView Component

ScrollView is a generic scrolling container that holds arbitrary components. Under the hood, iOS renders `UIScrollView` and Android renders `android.widget.ScrollView`.

### ❏ Critical Performance Warning

ScrollView renders ALL of its child components into memory immediately upon mounting. Never use ScrollView to display long, dynamic arrays or lists (e.g., 50+ items). Use FlatList instead, which virtualizes off-screen content.

## Technical Concepts & Architecture

- **style vs contentContainerStyle:** Maintains two style concepts: `style` applies to the outer frame boundary; `contentContainerStyle` applies to the scrollable content layout container inside.
- **Horizontal Axis:** Pass `horizontal={true}` to turn vertical scrolling into a horizontal carousel.
- **Pull to Refresh:** Supports native pull-to-refresh via the `` component.

## Key Props

- **contentContainerStyle:** Styles the scrollable inner child view container.
- **horizontal:** When true, content scrolls horizontally.



- **refreshControl**: Passes a `` component for pull-to-refresh.
- **showsVerticalScrollIndicator**: Hides vertical scrollbar.

#### ❑ ScrollView Example 1: Vertical Scroll with RefreshControl

```
import React, { useState } from 'react';
import { ScrollView, View, Text, RefreshControl, StyleSheet } from 'react-native';

export default function VerticalScrollViewWithRefresh() {
  const [refreshing, setRefreshing] = useState(false);
  const [items, setItems] = useState(['Alpha', 'Beta', 'Gamma', 'Delta', 'Epsilon']);

  const onRefresh = () => {
    setRefreshing(true);
    setTimeout(() => {
      setItems(['Alpha (Refreshed)', 'Beta', 'Gamma', 'Delta', 'Epsilon', 'Zeta']);
      setRefreshing(false);
    }, 1500);
  };

  return (
    <ScrollView
      style={styles.scrollFrame}
      contentContainerStyle={styles.scrollContent}
      refreshControl={
        <RefreshControl refreshing={refreshing} onRefresh={onRefresh} tintColor="#2563eb" />
      }
    >
      <Text style={styles.header}>Pull Down to Refresh</Text>
      {items.map((item, index) => (
        <View key={index} style={styles.card}>
          <Text style={styles.cardText}>{item}</Text>
        </View>
      ))}
    </ScrollView>
  );
}

const styles = StyleSheet.create({
  scrollFrame: { flex: 1, backgroundColor: '#f8fafc' },
  scrollContent: { padding: 20 },
  header: { fontSize: 18, fontWeight: 'bold', marginBottom: 16, color: '#1e3a8a' },
  card: { backgroundColor: 'white', padding: 16, borderRadius: 8, marginBottom: 10,
    borderWidth: 1, borderColor: '#e2e8f0' },
  cardText: { fontSize: 15, color: '#334155' },
});
```

The `RefreshControl` prop hooks into native drag-down-to-refresh gestures. When triggered, `onRefresh` sets `refreshing={true}` to show the native loading spinner until async data resolves.

#### ❑ ScrollView Example 2: Horizontal Card Carousel

```
import React from 'react';
import { ScrollView, View, Text, StyleSheet } from 'react-native';

export default function HorizontalCardCarousel() {
  const cards = ['Card #1', 'Card #2', 'Card #3', 'Card #4', 'Card #5'];

  return (
    <View style={styles.sectionContainer}>
      <Text style={styles.sectionTitle}>Horizontal Category List</Text>
    </View>
  );
}
```

```

    <ScrollView
      horizontal={true}
      showsHorizontalScrollIndicator={false}
      contentContainerStyle={styles.horizontalTrack}
    >
      {cards.map((card, idx) => (
        <View key={idx} style={styles.carouselCard}>
          <Text style={styles.carouselText}>{card}</Text>
        </View>
      ))}
    </ScrollView>
  </View>
);
}

const styles = StyleSheet.create({
  sectionContainer: { paddingVertical: 16 },
  sectionTitle: { fontSize: 16, fontWeight: 'bold', marginLeft: 20, marginBottom: 12 },
  horizontalTrack: { paddingHorizontal: 16, gap: 12 },
  carouselCard: {
    width: 130,
    height: 90,
    backgroundColor: '#2563eb',
    borderRadius: 10,
    justifyContent: 'center',
    alignItems: 'center',
  },
  carouselText: { color: 'ffffff', fontWeight: 'bold', fontSize: 14 },
});

```

Setting `horizontal={true}` changes the scroll axis. Setting `showsHorizontalScrollIndicator={false}` hides the bottom scroll bar for clean UI design.

#### ❏ **ScrollView Example 3: Mixed Form Content inside KeyboardAvoidingView**

```

import React from 'react';
import { ScrollView, View, Text, TextInput, KeyboardAvoidingView, Platform, StyleSheet } from
'react-native';

export default function FormInsideScrollView() {
  return (
    <KeyboardAvoidingView
      behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
      style={styles.flexContainer}
    >
      <ScrollView
        contentContainerStyle={styles.formContainer}
        keyboardShouldPersistTaps="handled"
      >
        <Text style={styles.title}>Settings Profile</Text>

        {Array.from({ length: 6 }).map((_, i) => (
          <View key={i} style={styles.inputGroup}>
            <Text style={styles.label}>Setting Field #{i + 1}</Text>
            <TextInput style={styles.input} placeholder={`Enter value ${i + 1}`} />
          </View>
        ))}
      </ScrollView>
    </KeyboardAvoidingView>
  );
}

```

```
const styles = StyleSheet.create({
  flexContainer: { flex: 1, backgroundColor: 'ffffff' },
  formContainer: { padding: 20 },
  title: { fontSize: 22, fontWeight: 'bold', marginBottom: 16 },
  inputGroup: { marginBottom: 14 },
  label: { fontSize: 13, fontWeight: '600', color: '#475569', marginBottom: 4 },
  input: { height: 44, borderWidth: 1, borderColor: '#cbd5e1', borderRadius: 8,
paddingHorizontal: 12 },
});
```

Combining `KeyboardAvoidingView` with `ScrollView` ensures input fields scroll cleanly into view above the virtual keyboard. `keyboardShouldPersistTaps='handled'` allows tapping buttons without needing to dismiss the keyboard first.

## 7. FlatList Component

FlatList is a high-performance virtualized list component (`RCTCollectionView` / `UITableView` on iOS and `androidx.recyclerview.widget.RecyclerView` on Android). It only renders list items currently visible inside the viewport window, recycling off-screen nodes to preserve RAM and GPU performance.

### Technical Concepts & Architecture

- **Virtualization Engine:** Only mounts items inside the viewport window plus a small buffer window. Unmounts off-screen items to maintain 60 FPS scrolling.
- **Data-Driven Layout:** Requires an array of items passed to `data` and a render function passed to `renderItem`.
- **Key Extraction:** Extracts unique string keys (`keyExtractor`) to optimize re-renders.
- **Infinite Scroll Pagination:** Fires `onEndReached` when user scrolls close to the bottom, enabling infinite scroll pagination.

### Key Props

- **data:** Array of data items to render.
- **renderItem:** Function rendering each row item: `{ item, index } => <Element />`.
- **keyExtractor:** Function returning a unique string key for each item.
- **ListHeaderComponent:** Header component pinned above list.
- **ListFooterComponent:** Footer component (e.g. loading spinner).
- **ListEmptyComponent:** Component rendered when data array is empty.
- **onEndReached:** Triggered when scroll distance reaches threshold.

#### FlatList Example 1: Basic Virtualized List with Selection State

```
import React, { useState } from 'react';
import { FlatList, View, Text, TouchableOpacity, StyleSheet } from 'react-native';

const mockData = Array.from({ length: 30 }, (_, i) => ({
  id: `item-${i + 1}`,
  title: `List Item #${i + 1}`,
  subtitle: `Description for item ${i + 1}`,
}));

export default function BasicVirtualizedList() {
```

```

const [selectedId, setSelectedId] = useState(null);

const renderItem = ({ item }) => {
  const isSelected = item.id === selectedId;
  return (
    <TouchableOpacity
      style={[styles.row, isSelected && styles.selectedRow]}
      onPress={() => setSelectedId(isSelected ? null : item.id)}
    >
      <Text style={[styles.title, isSelected && styles.selectedText]}>{item.title}</Text>
      <Text style={styles.subtitle}>{item.subtitle}</Text>
    </TouchableOpacity>
  );
};

return (
  <FlatList
    data={mockData}
    renderItem={renderItem}
    keyExtractor={(item) => item.id}
    ItemSeparatorComponent={() => <View style={styles.separator} />}
    contentContainerStyle={styles.listPadding}
  />
);
}

const styles = StyleSheet.create({
  listPadding: { padding: 16 },
  row: { padding: 16, backgroundColor: 'ffffff', borderRadius: 8, borderWidth: 1, borderColor: '#e2e8f0' },
  selectedRow: { backgroundColor: 'eff6ff', borderColor: '#2563eb' },
  title: { fontSize: 16, fontWeight: '600', color: '#0f172a' },
  selectedText: { color: '#2563eb' },
  subtitle: { fontSize: 13, color: '#64748b', marginTop: 2 },
  separator: { height: 8 },
});

```

The `FlatList` efficiently manages 30 items. Only items visible on screen are rendered in memory. `ItemSeparatorComponent` inserts spacing cleanly between rows.

### ❑ FlatList Example 2: Infinite Scroll Pagination & Lazy Loading

```

import React, { useState, useCallback } from 'react';
import { FlatList, View, Text, ActivityIndicator, RefreshControl, StyleSheet } from 'react-native';

const generatePage = (startIndex, count) =>
  Array.from({ length: count }, (_, i) => ({
    id: `${startIndex + i}`,
    name: `User User #${startIndex + i + 1}`,
  }));

export default function InfiniteScrollPagination() {
  const [data, setData] = useState(() => generatePage(0, 15));
  const [loadingMore, setLoadingMore] = useState(false);
  const [refreshing, setRefreshing] = useState(false);

  const loadMore = useCallback(() => {
    if (loadingMore) return;
    setLoadingMore(true);
  }, []);

```

```

      setTimeout(() => {
        setData((prev) => [...prev, ...generatePage(prev.length, 10)]);
        setLoadingMore(false);
      }, 1200);
    }, [loadingMore]);

    const onRefresh = useCallback(() => {
      setRefreshing(true);
      setTimeout(() => {
        setData(generatePage(0, 15));
        setRefreshing(false);
      }, 1200);
    }, []);

    return (
      <FlatList
        data={data}
        keyExtractor={({item}) => item.id}
        renderItem={({item}) => (
          <View style={styles.userCard}>
            <Text style={styles.userName}>{item.name}</Text>
          </View>
        )}
        onEndReached={loadMore}
        onEndReachedThreshold={0.5} // Trigger when user scrolls 50% from bottom
        refreshControl={<RefreshControl refreshing={refreshing} onRefresh={onRefresh} />}
        ListFooterComponent={() => loadingMore ? (
          <View style={styles.loaderFooter}>
            <ActivityIndicator size="small" color="#2563eb" />
            <Text style={styles.loaderText}>Loading more data...</Text>
          </View>
        ) : null}
      </FlatList>
    );
  }

  const styles = StyleSheet.create({
    userCard: { padding: 16, backgroundColor: 'ffffff', borderBottomWidth: 1, borderBottomColor: 'f1f5f9' },
    userName: { fontSize: 15, fontWeight: '500', color: '#1e293b' },
    loaderFooter: { padding: 16, alignItems: 'center', flexDirection: 'row', justifyContent: 'center', gap: 8 },
    loaderText: { fontSize: 13, color: '#64748b' },
  });

```

When the user scrolls within 50% of the list bottom (`onEndReachedThreshold={0.5}`), `onEndReached` triggers `loadMore()`, appending new data chunks dynamically.

#### ❑ FlatList Example 3: Multi-Column Grid Layout with Search & Empty State

```

import React, { useState } from 'react';
import { FlatList, View, Text, TextInput, StyleSheet } from 'react-native';

const products = [
  { id: '1', name: 'Wireless Mouse', price: '$29' },
  { id: '2', name: 'Mechanical Keyboard', price: '$89' },
  { id: '3', name: 'Gaming Monitor', price: '$299' },
  { id: '4', name: 'USB-C Hub', price: '$45' },
];

export default function MultiColumnGridWithSearch() {

```

```

const [search, setSearch] = useState('');

const filtered = products.filter((p) =>
  p.name.toLowerCase().includes(search.toLowerCase())
);

return (
  <View style={styles.container}>
    <TextInput
      style={styles.searchInput}
      placeholder="Filter products..."
      value={search}
      onChangeText={setSearch}
    />

    <FlatList
      data={filtered}
      keyExtractor={(item) => item.id}
      numColumns={2} // Two-column grid layout
      columnWrapperStyle={styles.columnRow}
      renderItem={({ item }) => (
        <View style={styles.gridCard}>
          <Text style={styles.productName}>{item.name}</Text>
          <Text style={styles.productPrice}>{item.price}</Text>
        </View>
      )}
      ListEmptyComponent={() => (
        <View style={styles.emptyBox}>
          <Text style={styles.emptyText}>No matching products found.</Text>
        </View>
      )}
    />
  </View>
);
}

const styles = StyleSheet.create({
  container: { flex: 1, padding: 16, backgroundColor: '#f8fafc' },
  searchInput: { height: 44, borderWidth: 1, borderColor: '#cbd5e1', borderRadius: 8,
paddingHorizontal: 12, marginBottom: 16, backgroundColor: 'ffffff' },
  columnRow: { justifyContent: 'space-between', marginBottom: 12 },
  gridCard: { width: '48%', padding: 16, backgroundColor: 'ffffff', borderRadius: 8,
borderWidth: 1, borderColor: '#e2e8f0' },
  productName: { fontSize: 14, fontWeight: 'bold', color: '#0f172a' },
  productPrice: { fontSize: 13, color: '#2563eb', marginTop: 4 },
  emptyBox: { padding: 40, alignItems: 'center' },
  emptyText: { color: '#94a3b8', fontSize: 14 },
});

```

Setting `numColumns={2}` transforms FlatList into a two-column grid. `ListEmptyComponent` automatically renders a clean empty state whenever `filtered` contains zero items.

## 8. SafeAreaView & StatusBar Component

SafeAreaView and StatusBar components handle system device chrome, ensuring app content renders safely around top camera notches, hole-punch cutouts, dynamic islands, and bottom gesture home bars.

### □ Industry Best Practice

React Native's legacy built-in `SafeAreaView` from 'react-native' only supports iOS and lacks edge control. Always install `react-native-safe-area-context` (`SafeAreaProvider`, `SafeAreaView`, `useSafeAreaInsets`) for complete Android & iOS notch protection.

## Technical Concepts & Architecture

- **System Inset Calculation:** Measures device hardware inset boundaries and applies automatic padding around notches and home indicator bars.
- **StatusBar Management:** Controls top system bar icon contrast (`light-content` for white icons, `dark-content` for black icons) and bar background colors.
- **Dynamic Insets Hook:** The `useSafeAreaInsets()` hook exposes pixel inset values (`top`, `bottom`, `left`, `right`) directly for custom header and footer styling.

### □ SafeArea Example 1: Standard Safe Area & Dynamic Theme Toggle

```
import React, { useState } from 'react';
import { View, Text, StatusBar, Switch, StyleSheet } from 'react-native';
import { SafeAreaProvider, SafeAreaView } from 'react-native-safe-area-context';

function ThemeSafeScreen() {
  const [isDark, setIsDark] = useState(false);

  return (
    <SafeAreaView style={[styles.container, isDark && styles.darkContainer]} edges={['top', 'bottom']}>
      <StatusBar
        barStyle={isDark ? 'light-content' : 'dark-content'}
        backgroundColor={isDark ? '#0f172a' : 'ffffff'}
      />

      <View style={styles.content}>
        <Text style={[styles.title, isDark && styles.darkText]}>Safe Area & Status Bar</Text>

        <View style={styles.toggleRow}>
          <Text style={[styles.label, isDark && styles.darkText]}>Dark Theme</Text>
          <Switch value={isDark} onChange={setIsDark} />
        </View>
      </View>
    </SafeAreaView>
  );
}

export default function App() {
  return (
    <SafeAreaProvider>
      <ThemeSafeScreen />
    </SafeAreaProvider>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: 'ffffff' },
  darkContainer: { backgroundColor: '#0f172a' },
  content: { padding: 20 },
  title: { fontSize: 22, fontWeight: 'bold', color: '#0f172a', marginBottom: 20 },
  darkText: { color: 'ffffff' },
  toggleRow: { flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center' },
  label: { fontSize: 16, fontWeight: '600' },
});
```

```
});
```

Wrapping the app in `` and using `

#### □ SafeArea Example 2: Dynamic Insets Hook (useSafeAreaInsets)

```
import React from 'react';
import { View, Text, StyleSheet } from 'react-native';
import { SafeAreaProvider, useSafeAreaInsets } from 'react-native-safe-area-context';

function DynamicInsetsContent() {
  const insets = useSafeAreaInsets(); // Returns { top, bottom, left, right }

  return (
    <View style={styles.screen, { paddingTop: insets.top, paddingBottom: insets.bottom }}>
      <View style={styles.headerBar}>
        <Text style={styles.headerText}>Custom Header (Notch Safe)</Text>
      </View>

      <View style={styles.body}>
        <Text style={styles.infoText}>Top Notch Height: {insets.top}px</Text>
        <Text style={styles.infoText}>Bottom Indicator Height: {insets.bottom}px</Text>
      </View>
    </View>
  );
}

export default function App() {
  return (
    <SafeAreaProvider>
      <DynamicInsetsContent />
    </SafeAreaProvider>
  );
}

const styles = StyleSheet.create({
  screen: { flex: 1, backgroundColor: '#f1f5f9' },
  headerBar: { height: 50, backgroundColor: '#1e3a8a', justifyContent: 'center', alignItems: 'center' },
  headerText: { color: 'ffffff', fontWeight: 'bold' },
  body: { padding: 20 },
  infoText: { fontSize: 15, color: '#334155', marginBottom: 8 },
});
```

The `useSafeAreaInsets()` hook gives exact pixel values for top notches and bottom indicators, allowing precise padding calculations for custom sticky headers or bottom bars.

#### □ SafeArea Example 3: Platform Translucent Status Bar Layout

```
import React from 'react';
import { View, Text, StatusBar, Platform, StyleSheet } from 'react-native';

export default function TranslucentAndroidStatusBar() {
  const androidStatusBarHeight = StatusBar.currentHeight || 0;

  return (
    <View style={styles.container}>
      <StatusBar translucent backgroundColor="transparent" barStyle="dark-content" />
    </View>
  );
}
```



```

    <View style={[styles.customHeader, { paddingTop: Platform.OS === 'android' ?
androidStatusBarHeight : 20 }]}>
      <Text style={styles.title}>Translucent Screen Header</Text>
    </View>

    <View style={styles.content}>
      <Text style={styles.body}>Header draws smoothly under the Android status bar.</Text>
    </View>
  </View>
);
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: 'ffffff' },
  customHeader: { backgroundColor: '#3b82f6', paddingHorizontal: 20, paddingBottom: 16 },
  title: { color: 'ffffff', fontSize: 20, fontWeight: 'bold' },
  content: { padding: 20 },
  body: { fontSize: 14, color: '#334155' },
});

```

On Android, setting `translucent backgroundColor='transparent'` allows background colors or images to render under the status bar, using `StatusBar.currentHeight` for exact padding.

## Summary Quick Reference Cheat Sheet

| Component    | Primary Use Case                         | Key Distinction / Caution                                                         |
|--------------|------------------------------------------|-----------------------------------------------------------------------------------|
| View         | Container layout & structural grouping   | Defaults to flexDirection: 'column'. Cannot directly contain raw string nodes.    |
| Text         | Displaying text & inline formatted spans | Only component allowed to contain raw text. Supports inline nesting.              |
| Image        | Local graphics & remote URL photos       | Remote URLs MUST have explicit width/height declared.                             |
| Button       | Quick primitive action triggers          | Cannot be styled with CSS style objects. Use TouchableOpacity for custom UI.      |
| TextInput    | Keyboard form inputs                     | Controlled state model. Use KeyboardAvoidingView to prevent keyboard obstruction. |
| ScrollView   | Scrolling small, fixed content blocks    | Renders ALL children at once. Unsuitable for long data lists.                     |
| FlatList     | Virtualized rendering of large lists     | Recycler rendering engine. Only mounts visible viewport rows.                     |
| SafeAreaView | Hardware notch & chrome padding          | Use react-native-safe-area-context for cross-platform Android & iOS support.      |